Desafio Backend — Guia para a Entrevista

O problema

- A API REST (Python) só sabe paginar **dentro de um módulo**: /modules/{id}/photos?offset=&limit=.
- O desafio pede uma única query GraphQL que pagine **a plataforma inteira**: 40 módulos × 2.000 fotos = 80.000 fotos.
- A página pode **atravessar a fronteira** entre dois módulos (ex.: offset 1990, limit 30 → 10 fotos do módulo 1 + 20 do módulo 2).
- Restrições: usar a REST (não gerar fotos no Kotlin), ordenar por módulo e foto, offset ≥ 0, limit entre 1 e 200 (padrões 0 e 50), página vazia quando offset passa do total, erros claros, **não carregar as 80.000 fotos**, e ter testes.

Arquitetura: as camadas

```
Cliente GraphQL
   │  query platformPhotos(platformId, offset, limit)
   ▼
esquema.graphqls          → o "contrato": o que o cliente pode pedir
   ▼
PlatformPhotoDataFetcher  → liga a query ao Kotlin (DGS); não tem lógica
   ▼
PlatformPhotoService      → validação + paginação global + tradução de erros
   ▼
RestApiClient (já existia)→ faz as chamadas HTTP
   ▼
API REST Python
```

Arquivo	Responsabilidade
esquema.graphqls	Define os tipos Query, PlatformPhotoPage e Photo.
PlatformPhotoDataFetcher.kt	@DgsComponent + @DgsQuery: recebe os argumentos e chama o Service.
PlatformPhotoService.kt	Toda a regra de negócio: validação, cálculo da página, chamadas REST, erros.
FakeRestApi.kt (teste)	Servidor falso que imita a REST e conta as chamadas recebidas.
PlatformPhotoServiceTest.kt	Testa a lógica: casos do enunciado, erros, nº de chamadas.
PlatformPhotoDataFetcherTest.kt	Testa a query GraphQL de ponta a ponta (padrões e formato dos erros).

Por que separar DataFetcher e Service? O DataFetcher só "traduz" GraphQL para Kotlin. A lógica fica no Service, que pode ser testado sem subir GraphQL. Se amanhã alguém quiser expor a mesma regra por REST, reaproveita o Service. É o princípio de responsabilidade única.

O schema GraphQL

```
type Query {
    platformPhotos(platformId: Int!, offset: Int! = 0, limit: Int! = 50): PlatformPhotoPage!
}

type PlatformPhotoPage {
    items: [Photo!]!
    offset: Int!
    limit: Int!
    total: Int!
    hasNextPage: Boolean!
}

type Photo { id: Int!  moduleId: Int!  name: String!  url: String! }
```

Decisões que você deve saber explicar

- **Uma única query** (platformPhotos): o enunciado pede uma operação só. Internamente ela pode fazer várias chamadas REST.
- **`!` (não nulo)**: todos os campos sempre existem, então o cliente não precisa tratar null. Em [Photo!]!, nem a lista nem os itens são nulos (uma página vazia é [], não null).
- **`Int! = 0` e `Int! = 50`**: o argumento é opcional (usa o padrão se omitido), mas não aceita null explícito. Assim o Kotlin sempre recebe um número.
- **`Int` em vez de `ID`**: os IDs da REST são inteiros (1 a 40, 1 a 1600…). ID no GraphQL é serializado como texto; Int deixa o contrato mais simples. Ambos seriam aceitáveis — o importante é saber justificar.
- **Metadados da página**: offset, limit, total e hasNextPage são o que o enunciado pede para o cliente conseguir continuar paginando (próxima página = offset + limit).
- **Documentação no schema**: os textos entre aspas aparecem no GraphiQL para quem usa a API.

O algoritmo de paginação (a parte mais importante)

A ideia central

Imagine as 80.000 fotos enfileiradas. Como **cada módulo tem exatamente 2.000 fotos**, a posição global de uma foto diz onde ela está:

- posição / 2000 (divisão inteira) → **qual módulo** (índice na lista de módulos).

- posição % 2000 (resto) → **onde dentro do módulo**.

Posição global	posição / 2000	posição % 2000	Significa
0	0	0	1ª foto do 1º módulo
1990	0	1990	foto 1991 do 1º módulo
2000	1	0	1ª foto do 2º módulo
79990	39	1990	foto 1991 do 40º módulo

O código, passo a passo

```
val total = modules.size * PHOTOS_PER_MODULE                // 40 * 2000 = 80.000
val end = offset + minOf(limit, maxOf(total - offset, 0))    // onde a página termina
val items = mutableListOf<Photo>()                           // "sacola" das fotos
var position = offset                                       // cursor

while (position < end) {
    val module = modules[position / PHOTOS_PER_MODULE]       // qual módulo
    val offsetInModule = position % PHOTOS_PER_MODULE        // onde dentro dele
    val count = minOf(end - position, PHOTOS_PER_MODULE - offsetInModule)
    val page = restApi.photos(module.id, offsetInModule, count)
    if (page.items.isEmpty()) break
    items += page.items
    position += page.items.size
}
hasNextPage = offset + items.size < total
```

Variável	O que é
total	Total de fotos da plataforma: nº de módulos × 2000.
end	Posição onde a página termina (sem incluir). Normalmente offset + limit, mas **nunca passa do total**: minOf escolhe o menor entre o pedido e o que existe; maxOf(…, 0) evita número negativo quando o offset já passou do total.
items	Lista mutável onde as fotos de cada chamada são juntadas. Necessária porque a página pode vir de dois módulos.
position	Cursor: a próxima posição global que ainda falta buscar. Avança pelo número de fotos que realmente chegaram.
count	Quantas fotos pedir neste módulo: o que falta para a página OU o que sobra no módulo — o menor dos dois.
break	Proteção: se a REST devolver lista vazia, sai do laço. Sem isso, position não avançaria e o laço seria infinito.

Os 4 casos do enunciado (saiba fazer de cabeça)

offset / limit	end	Chamadas REST de fotos	Resultado
0 / 50	50	módulo 1: offset 0, limit 50	50 fotos do módulo 1, hasNextPage = true
1990 / 30	2020	módulo 1: offset 1990, limit 10 módulo 2: offset 0, limit 20	10 do módulo 1 + 20 do módulo 2
79990 / 50	80000	módulo 40: offset 1990, limit 10	últimas 10 fotos, hasNextPage = false
80000 / 50	80000	nenhuma (o laço nem roda)	página vazia, hasNextPage = false

Quantidade de chamadas REST

- **1 chamada** para /platforms/{id}/modules — precisa dos IDs dos módulos e também serve para saber se a plataforma existe.
- **No máximo 2 chamadas** de fotos: como limit ≤ 200 e um módulo tem 2.000 fotos, uma página nunca cobre mais de dois módulos.
- **Total: no máximo 3 chamadas**, independente do offset. Nunca carrega as 80.000 fotos.
- Por que não usar /platforms para validar a plataforma? Seria uma chamada a mais; o 404 de /modules já responde isso.

Tratamento de erros

Situação	Onde é detectada	Erro GraphQL (errorType)
offset < 0 ou limit fora de 1–200	Validação no início do Service, **antes** de chamar a REST	BAD_REQUEST
Plataforma inexistente (ex.: 41)	REST responde 404 em /modules → HttpClientErrorException.NotFound	NOT_FOUND
REST fora do ar, erro 500, timeout	Qualquer RestClientException	UNAVAILABLE (exceção própria RestApiUnavailableException)
Offset além do total	Não é erro: retorna página vazia	—

Testes

- **FakeRestApi**: servidor HTTP embutido (do próprio JDK) que imita a REST Python e **registra cada requisição**. Por isso os testes não dependem do Python rodando e conseguem **provar quantas chamadas REST foram feitas**. Também simula a REST quebrada (failing = true → 500).
- **PlatformPhotoServiceTest** (8 testes): os 4 casos do enunciado; plataforma 2 usa os módulos certos (41…); parâmetros inválidos rejeitados **sem chamar a REST**; plataforma inexistente; falha da REST; e as URLs exatas chamadas no caso da fronteira.
- **PlatformPhotoDataFetcherTest** (2 testes): executa a query GraphQL de verdade com DgsQueryExecutor — confere os valores padrão (offset 0, limit 50) e que os erros chegam com errorType e mensagem.
- Total do projeto: **12 testes passando** (incluindo os 2 que já existiam).

Como rodar: cd graphql-backend e depois .\gradlew.bat test (Windows)

Como executar (para demonstrar ao vivo)

1. Terminal 1, na raiz: `python rest-api/server.py` .
2. Terminal 2: `cd graphql-backend` e `.\gradlew.bat bootRun`. Fica em "85% EXECUTING" — é normal, o servidor está no ar.
3. Abra http://localhost:8080/graphiql e execute a query abaixo; troque o offset para 0, 79990 e 80000.
4. Testes: `.\gradlew.bat test`.

```
{
 platformPhotos(platformId: 1, offset: 1990, limit: 30) {
  offset limit total hasNextPage
  items { id moduleId name url }
 }
}
```